# Contents

# Chapter 0

# Introduction

Parametric polymorphism provides a language-independent abstraction, its implementation varies considerably among programming languages and compiler architectures.

The principal design choice concerns how generic types are represented after compilation and when concrete type instantiations are produced.

These implementation strategies influence several aspects of the execution model, including runtime representation, reflection capabilities, performance, and binary size.

The following dimensions constitute the primary axes along which implementations of parametric polymorphism are typically compared.

## 0.1 Instantiation Time

A first distinguishing characteristic is the stage at which generic definitions are instantiated.

Some implementations perform instantiation entirely during compilation.

In this approach, each concrete use of a generic function or data structure is expanded into a specialized implementation before executable code is generated.

Since all specializations already exist in the final binary, runtime execution incurs no additional generic instantiation cost.

Other implementations compile generic definitions only once.

Generic type parameters are verified during type checking and subsequently removed or replaced by a common representation.

Consequently, no additional executable code is generated for different type arguments, and all instantiations share the same compiled implementation.

## 0.2 Runtime Representation

Implementations also differ in how generic types are represented after compilation.

*Type erasure* removes generic type parameters, producing a homogeneous representation where different instantiations share the same executable code.

The runtime system therefore treats different generic instantiations as equivalent representations.

Conversely, *reified generics* preserve concrete type information by generating distinct specialized code for each instantiated type.

Each generic instantiation may therefore possess its own runtime representation, memory layout, and executable instructions.

## 0.3 Reflection and Runtime Type Information

The availability of generic type information at runtime depends directly on the chosen representation strategy.

When type erasure is employed, generic parameters are generally unavailable after compilation.

Reflection mechanisms therefore provide limited access to instantiated generic types because most type information has been discarded.

Conversely, implementations based on reified generics preserve concrete type information throughout compilation.

*Runtime reflection or runtime type identification (RTTI)* may therefore recover the actual instantiated types, enabling operations such as generic type inspection, specialized serialization, and runtime metadata analysis.

## 0.4 Type Safety

Both implementation strategies preserve static type safety, although they achieve this objective differently.

In implementations based on type erasure, generic correctness is verified entirely during compilation.

Although type parameters disappear from the generated executable, compiler-inserted casts preserve the guarantees established by the static type system.

Implementations based on reified generics retain complete type information throughout compilation and generate type-specific implementations directly.

Since each specialization corresponds to a concrete type, correctness is preserved without requiring erased representations or implicit runtime casts.

## 0.5 Performance

The implementation strategy significantly influences both compilation and execution performance.

Homogeneous implementations generally produce smaller binaries because generic definitions are compiled only once; compilation time is often reduced as well, although this benefit can be partially offset by the additional work required to insert runtime casts, boxing/unboxing operations, or bridge methods.

However, the shared representation may require additional indirections, boxing, or runtime casts, limiting optimization opportunities.

Heterogeneous implementations incur greater compilation cost and larger executables because every instantiated type generates a distinct implementation.

Nevertheless, the compiler can aggressively optimize each specialization, exploiting concrete type information for improved instruction selection, memory layout, inlining, and register allocation.

Consequently, runtime performance often approaches that of manually specialized code.

## 0.6 Summary

The implementation of parametric polymorphism is primarily characterized by five orthogonal dimensions:

1. the stage at which generic instantiation occurs;
2. the runtime representation of generic types;
3. the availability of runtime type information and reflection;
4. the trade-off between compilation cost, binary size, and execution performance;
5. the mechanisms through which static type safety is preserved.

These dimensions form the basis for comparing homogeneous implementations based on type erasure with heterogeneous implementations based on reification or monomorphization.

# Chapter 1

# C++ Templates

C++ implements parametric polymorphism through the *template* mechanism, a compile-time construct that allows the definition of generic classes and functions parameterized over one or more types. Rather than producing a single generic runtime entity, the compiler generates an independent specialized implementation for each concrete type argument used by the program, a process known as monomorphization. This chapter examines when this instantiation occurs, how it shapes the runtime representation of specialized code, and the consequences this reified model has for performance, static type safety, and the limited runtime type information exposed through RTTI.

## 1.1 Instantiation Time

C++ implements parametric polymorphism through the *template* mechanism, which adopts a purely compile-time instantiation model. A template is not itself an executable entity but a parameterized blueprint from which the compiler generates independent concrete implementations. Executable code is generated only after the complete set of template arguments is known, so all instantiation is resolved during static compilation and no instantiation mechanism exists at runtime.

### 1.1.1 Template Definition

When the compiler encounters a template definition, it performs the front-end stages (lexical analysis, parsing, and the semantic checks possible without knowledge of future arguments) and stores a parameterized internal representation, without generating executable code:

```
template<typename T>
T get_max(T a, T b)
{
    return (a > b) ? a : b;
}
```

Since the compiler still ignores the concrete meaning of `T`, it cannot yet determine the operations, layout, or instructions required by the final implementation; the definition remains a blueprint awaiting instantiation.

### 1.1.2 Instantiation Trigger

Instantiation begins when the compiler encounters a concrete use of the template, either via explicit argument specification or implicit type deduction:

**Example.**

```
int i = get_max(3, 7);
double d = get_max<double>(2.5, 4.8);
```

The first invocation relies on deduction ($T = int$); the second specifies the argument explicitly ($T = double$). The compiler generates two distinct instantiations: `get_max<int>` and `get_max<double>`.

Each unique set of arguments constitutes a distinct instantiation, triggered through *implicit instantiation* (performed automatically when required), *explicit instantiation* (requested directly by the programmer), or *explicit specialization* (a wholly different implementation provided for a specific argument combination). In most cases, code generation begins once the complete set of arguments is known; the exception is the explicit instantiation declaration (extern template), which suppresses code generation in the current translation unit even though the arguments are fully known, deferring it to a translation unit containing the corresponding explicit instantiation definition.

### 1.1.3 Template Argument Substitution

Once triggered, the compiler substitutes every formal parameter with its concrete argument:

**From previous example.** $T \to int$ produces:

```
int get_max(int a, int b)
{
    return (a > b) ? a : b;
}
```

while $T \to double$ produces the analogous specialization for `double`.

From this point, the compiler treats each specialization as if it had been written explicitly, behaving as an ordinary non-template function.

### 1.1.4 Semantic Analysis of the Specialization

Instantiation does not immediately produce executable code: each generated specialization first undergoes the normal semantic analysis applied to ordinary C++ entities, including type checking, overload resolution, name lookup, constraint verification, constant expression evaluation, and instantiation of nested templates. This step is essential because many semantic properties cannot be verified before substitution occurs; consequently, a syntactically correct template may still fail to compile if a specific instantiation violates the requirements imposed by the template body.

### 1.1.5 Intermediate Representation and Code Generation

Once semantic analysis succeeds, the specialization enters the compiler back-end, which generates an Intermediate Representation, applies the standard optimization pipeline (constant propagation, dead-code elimination, inlining, loop optimizations, register allocation, instruction selection), and emits native machine code exclusively for that specialization. This process repeats independently for every distinct instantiation, so each specialization owns an independent executable implementation; no shared representation is produced that could serve multiple template arguments.

### 1.1.6 Completion of the Instantiation Process

Instantiation is triggered and resolved entirely during compilation: by the time object files are generated, every specialization required by a given translation unit already exists as compiled code. When the same instantiation is required by multiple translation units, each one generates its own copy, and it is the linker that discards the duplicates and retains a single definition in the final executable. The lifecycle can be summarized as: parsing the definition, storing it as a parameterized representation, triggering instantiation on concrete use, substituting arguments, performing semantic analysis, generating IR, optimizing, and emitting native code included in the executable before execution begins. The defining characteristic of C++ parametric polymorphism is therefore that the entire process completes during static compilation; when execution begins, no

instantiation mechanism remains active, and the runtime simply executes machine code already generated for every required specialization.

## 1.2 Runtime Representation

C++ templates transform generic source-level abstractions into concrete program entities before execution. A template declaration does not itself correspond to a runtime object; it defines a family of possible entities materialized only when the compiler generates a specific specialization. The runtime representation of a specialization is therefore identical to that of an ordinary function, class, or object: after linking, the executable contains native instructions and data associated with each instantiated specialization, and the loader and processor have no knowledge of the original template or its parameters. Conceptually, a template is a compile-time transformation $T : (A_1, ..., A_n) \rightarrow P$, so that $T<int> \rightarrow P_{int}$ and $T<double> \rightarrow P_{double}$ are independent implementations; at runtime the program never invokes a generic representation of $T$, only the machine code generated for each concrete specialization.

### 1.2.1 Runtime Form of Template Specializations

Consider the following function template:

```
template <typename T>
T add(T a, T b)
{
    return a + b;
}
```

When the program requires `add<int>(1,2)` and `add<double>(1.0,2.0)`, the compiler produces two separate functions, `add<int>(int,int)` and `add<double>(double,double)`, represented at the binary level as independent machine-code regions with their own instructions, symbol identity, and entry address. The runtime system contains no generic function body carrying a runtime type parameter, only the concrete implementations produced during translation; control flow transfers directly to the generated addresses, and the processor executes without any awareness that the code originated from a template. Hence the runtime representation of add<int> and add<double> is not a single generic function add<T>, but two independent concrete functions produced before execution.

### 1.2.2 Monomorphization

C++ uses heterogeneous translation, a process known in the C++ standard and literature as template instantiation and sometimes informally referred to, by analogy with other languages such as Rust, as monomorphization. For example:

```
template <typename T>
struct Container
{
    T value;
};

Container<int> a;
Container<float> b;
```

creates two independent specializations, `Container<int>` and `Container<float>`; the executable contains their concrete representations, with no runtime object representing `Container<T>` since the generic abstraction was already resolved during translation. For function templates, each specialization produces a distinct machine-code artifact behaving exactly like a function written explicitly for that type, with its own entry point invoked through normal control-flow instructions.

### 1.2.3 Name Mangling

Compilers encode template arguments into symbol names through *name mangling*, allowing different specializations to coexist in object files and letting the linker distinguish entities such

as `function<int>()` and `function<double>()`; ;the exact mangling scheme is not standardized across compilers b t is defined by the underlying ABI, such as the Itanium C++ ABI used by GCC and Clang.

### 1.2.4 Final Executable Representation

After linking, the executable is a collection of concrete compiled entities, $Executable = \{Code(P_1), ..., Code(P_n)\}$, where each $P_i$ corresponds to a template specialization. It contains machine instructions for instantiated functions, symbols identifying them, relocation information, and optionally runtime metadata produced by other C++ mechanisms (Section 1.5) – artifacts no different from those produced by non-template code. During execution the processor never interprets template parameters: a call instruction transfers control to a compiled function's address, and the link back to the original template source survives only in debugging information or symbol naming conventions, not in the runtime execution model.

### 1.2.5 Absence of Generic Metadata

A defining characteristic of the C++ model is the absence of runtime generic metadata: after instantiation, the executable retains no information describing the original template declaration or its possible arguments. Given `template<typename T> void process(T value);`, the runtime environment only observes concrete functions such as `process<int>()` or `process<double>()` – template declarations, parameter lists, generic type variables, and mechanisms for creating new specializations dynamically simply do not exist as runtime entities; the execution environment manipulates only the final native representation. The limited runtime type identification C++ does provide is discussed in Section 1.5.

## 1.3 Performance Characteristics

The performance characteristics of C++ templates derive directly from monomorphization (Section 1.2.2): every instantiation is transformed into an independent specialization operating on a concrete type, so the executable contains ordinary statically typed functions and classes tailored to each instantiated type rather than a universal generic implementation. For example:

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

instantiated with different types generates specialized functions equivalent to handwritten `int max(int,int)` and `double max(double,double)` versions, whose machine code operates directly on concrete data types without runtime mechanisms for resolving the generic parameter.

### 1.3.1 Static Dispatch and Elimination of Runtime Overhead

One of the main performance advantages of monomorphization is static dispatch: since the concrete type of every parameter is known at compile time, function calls are resolved before execution, eliminating the indirections required by dynamic dispatch. Given `template<typename T> void process(T& object) { object.compute(); }` called with a `Matrix`, the compiler ù resolves the type of object statically at compile time; if `Matrix::compute()` is not virtual, the call itself is also resolved directly, reducing overhead and exposing further optimization opportunities, whereas a `virtual compute()` would still be dispatched through the virtual table at runtime despite the type being known at compile time. A related consequence is the elimination of boxing and unboxing: template-generated code operates directly on the native representation of values, so `std::vector<int>` stores integers in their native form without conversion into a generic object wrapper.

### 1.3.2 Compiler Optimization Opportunities

Because every specialization is fully typed, optimizing compilers apply the same techniques used for manually written specialized code. A fundamental example is function inlining: since both implementation and argument types are known at compile time, `template<typename T> inline T square(T x) { return x*x; }` called as `square(5)` can, at the compiler's discretion, be replaced directly by `5*5`, removing call overhead and enabling further constant propagation, constant folding, dead-code elimination, loop optimization, and strength reduction; the `inline` keyword only permits this substitution by affecting linkage, without guaranteeing that the compiler will actually perform it. Template parameters can also encode compile-time constants – e.g. a `template<int N>` array bound lets the compiler know the exact iteration count of a loop and apply loop unrolling. Compile-time specialization also enables branch elimination via `if constexpr`, where only the branch matching the selected type is emitted, and creates the preconditions for automatic vectorization: because concrete types and memory layouts are known, numerical templates such as an element-wise array `add` become eligible for compilation into SIMD instructions, although whether vectorization actually occurs still depends on the optimizer, the compilation flags, and the absence of pointer aliasing between operands.

### 1.3.3 Zero-Overhead Abstraction and Trade-offs

The combination of static dispatch, direct data representation, and compiler optimization enables C++ templates to follow the *zero-overhead abstraction* principle. According to this principle, abstractions introduced at the source level should not impose additional runtime costs compared with an equivalent manually specialized implementation. Template-based abstractions can therefore provide generic programming capabilities while maintaining execution efficiency comparable to explicitly written type-specific code.

The main performance limitation associated with template monomorphization is related to the expansion of generated code. Since every specialization produces an independent implementation, extensive template usage may increase executable size. A larger instruction footprint can negatively affect instruction cache locality, increasing instruction cache misses in performance-critical applications.

This effect does not originate from additional runtime computation introduced by templates, but from the interaction between generated specialized code and the processor memory hierarchy. Therefore, template performance is often dominated by the benefits of compile-time specialization, while instruction cache behavior represents the main architectural factor capable of reducing these advantages; in codebases with extensive template usage over many distinct types, however, the resulting code bloat can also affect compilation and linking time, and in some cases can outweigh the benefits of specialization rather than merely reducing them.

## 1.4 Type Safety

Static type safety represents one of the fundamental objectives of parametric polymorphism. Its purpose is to guarantee that every operation performed on a value is compatible with its statically known type before program execution. In this way, incorrect type interactions are detected during compilation rather than being deferred to runtime.

Different implementations of parametric polymorphism achieve static type safety through different mechanisms, as introduced in the overview of implementation strategies. C++ templates adopt a specialization-based implementation strategy based on the template instantiation and monomorphization process described in Section 1.2.2: type parameters are never erased before compile-time checking, and every instantiation is checked against a concrete, fully specialized implementation, even though no corresponding type information survives as a runtime entity.

From a type system perspective, reification allows the compiler to retain complete information about the actual types involved in every specialization. Unlike erased representations, where the original type parameter is no longer available during execution, C++ templates preserve

the relationship between the generic definition and the concrete types used to instantiate it. Consequently, all type-dependent operations are checked directly against the substituted types.

### 1.4.1 Verification of Template Correctness

The verification of template correctness occurs through the ordinary C++ static type system. After template argument substitution, the compiler performs semantic analysis on the generated specialization, checking that all expressions, operators, conversions, and function calls are valid for the concrete type. If a required operation is not supported by the instantiated type, the specialization is considered ill-formed and the program is rejected during compilation.

This mechanism differs fundamentally from implementations based on type erasure because C++ templates never require the compiler to recover discarded type information. Since each specialization is generated from a known concrete type, operations are resolved statically according to the rules of the language type system. Therefore, no implicit runtime casts are required to restore the original generic type information.

### 1.4.2 Example: Type Checking Across Specializations

The following example illustrates how C++ templates preserve static type safety through concrete specialization.

```
#include <string>

template <typename T>
T maximum(const T& a, const T& b)
{
    return (a < b) ? b : a;
}

int main()
{
    int x = maximum(3, 7);

    double y = maximum(3.5, 8.1);

    // Compilation error:
    // no valid specialization exists because
    // the arguments do not have the same type.

    // auto z = maximum(3, std::string("hello"));
}
```

The function template `maximum` describes a generic operation that can be applied to any type satisfying the required semantic constraints. When the compiler encounters the expression `maximum(3, 7)`, template argument deduction determines that the parameter `T` corresponds to the type `int`. The compiler then creates a specialization equivalent to:

```
int maximum(const int& a, const int& b);
```

Similarly, the invocation `maximum(3.5, 8.1)` produces a specialization where `T = double`, and the generated function operates exclusively on values of type `double`.

Each specialization is independently checked by the compiler. The comparison operator `<` must exist for the instantiated type, the return expression must be compatible with the declared return type, and all parameter bindings must satisfy the requirements imposed by the C++ type system.

The invocation `maximum(3, std::string("hello"))`, shown commented out above so that the rest of the example still compiles, would not produce a valid specialization if enabled: template argument deduction cannot identify a single type for `T`, so the compiler would reject the program before executable code is generated. No runtime type inspection, dynamic cast, or conversion mechanism is involved, because the type inconsistency is detected entirely during static analysis.

### 1.4.3 Type Safety Across Instantiations

A further consequence of template reification is that type checking is performed with respect to each concrete template instantiation. A template definition represents a family of possible programs, while every generated specialization represents a fully specified program whose types are known. Therefore, correctness is established separately for every instantiated version rather than through a single erased generic representation.

This property means that template abstraction does not weaken the C++ static type system. Instead, templates delay the determination of concrete types until instantiation, after which normal compile-time type checking is applied to the resulting program. Every successfully generated specialization is consequently an ordinary statically typed C++ entity, whose correctness has been verified before execution.

From a type-theoretic perspective, C++ templates achieve static type safety through reification: generic definitions preserve type information, template arguments are substituted with concrete types, and the resulting specializations are validated according to the complete rules of the language type system. This approach guarantees generic correctness without requiring erased representations or compiler-generated runtime casts.

## 1.5 Reflection and Runtime Type Information

As introduced in the overview of implementation strategies, the availability of generic type information at runtime depends directly on whether the implementation follows a type-erasure or a reified model. C++ templates follow neither model in the runtime sense: template parameters are resolved through monomorphization described in Section 1.2.2, which preserves the identity of the concrete types used during compilation, but this identity is not represented as a runtime generic entity once compilation is complete. However, compile-time type preservation should not be confused with runtime reflection. After compilation, the C++ execution environment does not generally provide a universal mechanism capable of enumerating arbitrary template parameters, inspecting source-level declarations, or dynamically querying all properties of instantiated template entities. What C++ does provide is a restricted mechanism, Run-Time Type Information (RTTI), which allows limited runtime identification of concrete types.

### 1.5.1 The `typeid` Operator and `std::type_info`

The primary standard facilities for RTTI are the operators `typeid` and `dynamic_cast`, together with the `std::type_info` class.

The `typeid` operator obtains runtime type information associated with an expression or a type. When applied to an instantiated template class, it can identify the concrete specialization generated by the compiler. For example:

```
template <typename T>
class Wrapper {
};

Wrapper<int> a;
Wrapper<double> b;

std::cout << typeid(a).name();
std::cout << typeid(b).name();
```

The two objects correspond to different C++ types, namely `Wrapper<int>` and `Wrapper<double>`. Consequently, the RTTI system treats them as distinct runtime types whenever RTTI metadata is generated for these entities. The resulting `std::type_info` objects represent the runtime identity of the instantiated classes.

The information exposed by `std::type_info` is intentionally limited. It allows comparison of run-

time type identities through operations such as equality comparison and provides an implementation-defined textual representation through `name()`. It does not provide access to template arguments, class members, inheritance hierarchies, or source-level metadata in the manner of a full reflection system.

### 1.5.2 `dynamic_cast` and Polymorphic Template Classes

The `dynamic_cast` operator provides another RTTI mechanism for performing safe runtime conversions within polymorphic class hierarchies. A class must contain at least one virtual function for `dynamic_cast` to succeed, and for typeid applied to an expression to report the dynamic type rather than the static type known at compile time.

Template classes can participate in such hierarchies like ordinary C++ classes. For example:

```
class Base {
    public:
    virtual ~Base() = default;
};

template <typename T>
class Derived : public Base {
};

Base* ptr = new Derived<int>();

Derived<int>* result = dynamic_cast<Derived<int>*>(ptr);
```

During execution, `dynamic_cast` uses runtime type information associated with the object to verify whether the actual object type corresponds to the requested target type. In this example, the cast succeeds because the dynamically stored object is an instance of the concrete template specialization `Derived<int>`.

The same operation would fail for an object instantiated with a different template argument, such as `Derived<double>`, because each template specialization represents a distinct C++ type. Therefore, RTTI preserves the identity of instantiated template classes when those classes are part of a polymorphic hierarchy.

### 1.5.3 Limits of RTTI as a Reflection Mechanism

C++ templates demonstrate a hybrid relationship between type preservation and runtime identification. The template mechanism preserves concrete type information during compilation through specialization generation, but this information is not automatically exposed through a general runtime reflection interface.

The compiler knows the complete instantiated type during template compilation. It can therefore perform static analysis, select specialized implementations, and generate code based on the concrete template arguments. This information belongs to the compile-time type system and is consumed before program execution.

RTTI, in contrast, represents a restricted runtime mechanism. It operates only on the subset of type information explicitly retained by the C++ object model, primarily for polymorphic objects and explicit type identification through `typeid`. For example, a template class such as:

```
template<typename T>
class Base
{
public:
    virtual ~Base(){}
};
```

```
Base<int> object;
```

may generate runtime type information describing `Base<int>`, but it does not generate a runtime descriptor describing the generic definition `Base<T>`, because the generic template abstraction has no runtime existence (Section 1.2).

Consequently, RTTI can distinguish between instantiated template types when the compiler emits the corresponding runtime metadata, but it cannot reconstruct the complete template definition or inspect arbitrary template parameters dynamically. C++ templates therefore preserve generic type information through compile-time monomorphization rather than through runtime reification, and standard RTTI mechanisms provide runtime access to the identity of concrete instantiated types without constituting a complete reflection system.

# Chapter 2

# Java Generics

Java implements parametric polymorphism through *generics*, a compile-time mechanism that allows classes and methods to be parameterized over types while relying on *type erasure* rather than specialization: generic type arguments are verified statically and then removed from the compiled representation, so that all parameterizations of a generic declaration share a single runtime implementation. This chapter discusses when this instantiation and erasure take place, what runtime representation results from it, and how this erased model affects execution performance, the preservation of static type safety, and the limits it imposes on runtime reflection.

## 2.1 Instantiation Time

Java implements parametric polymorphism through a compile-time instantiation model based on *type erasure*. During compilation, the compiler associates concrete type arguments with generic declarations for the sole purpose of static type checking; this association does not, however, produce a specialized runtime class. This contrasts directly with C++, where template instantiation triggers monomorphization and generates an independent specialization for every concrete type argument.

### 2.1.1 Generic Declaration

Consider the following generic class:

```
class Box<T> {
    private T value;
    public T get() { return value; }
}
```

The compiler stores a single parameterized representation; unlike C++ templates, no family of specializations will later be generated from it.

### 2.1.2 Instantiation Trigger

Instantiation occurs when the compiler encounters a concrete parameterized use:

**Example.**

```
Box<String> box = new Box<>();
Box<Integer> numbers = new Box<>();
```

The compiler associates $T = String$ and $T = Integer$ respectively for type-checking purposes. Unlike the C++ case, no distinct classes `Box<String>` or `Box<Integer>` exist afterward; both instantiations resolve to the same erased implementation.

### 2.1.3 Type Erasure Pipeline

After compile-time verification, the compiler applies type erasure: an unbounded parameter `<T>` is replaced by `Object`, while a bounded parameter `<T extends Number>` is replaced by the erasure of its leftmost bound:

**From previous example.**

```
class Box {
    private Object value;
    public Object get() { return value; }
}
```

At this point, no additional class corresponding to `Box<String>` exists; the generic parameter has been discarded from the representation.

### 2.1.4 Compiler-Inserted Casts

Because generic parameters are removed before execution, the compiler inserts casts to preserve the semantics established during instantiation. The expression `String s = box.get();` compiles to a call returning `Object`, followed by an inserted `checkcast` instruction.

### 2.1.5 Completion of the Instantiation Process

Instantiation terminates entirely during compilation: type parameters are verified statically, then erased before bytecode generation. The defining characteristic of Java parametric polymorphism is therefore that instantiation is exclusively a compile-time verification step resolving to a single shared runtime representation, in direct contrast to the per-argument specialization performed by C++ monomorphization.

## 2.2 Runtime Representation

Java generics are implemented through type erasure: generic type parameters are removed from the executable representation before runtime, and different parameterizations of the same generic class share a single runtime implementation. Source-level types `List<String>` and `List<Integer>` therefore correspond to the same JVM runtime class, `List`, with no separate metadata, layout, or method implementation maintained per argument.

### 2.2.1 Erasure of Generic Parameters

Given `class Box<T> { T value; T get(); }`, erasure produces `class Box { Object value; Object get(); }`. Formally, $Box\langle T\rangle \to Box\langle Object\rangle$ for an unbounded variable, so $RuntimeType(Box\langle Str$ $RuntimeType(Box\langle Integer\rangle) = Box$: the generic parameter plays no role in runtime type identity.

### 2.2.2 Object Layout and Storage

Since no specialized classes are generated, a generic field `T value` is represented after erasure as `Object value`, storing a reference rather than a value specialized for the concrete argument. `ArrayList<String>` and `ArrayList<Integer>` therefore share an identical internal structure, unlike C++ monomorphization, where each instantiation may produce a distinct object layout.

### 2.2.3 Inserted Runtime Casts

Retrieval from a generic structure requires an inserted cast: `String s = list.get(0);` becomes `Object tmp = list.get(0); String s = (String) tmp;`, executed via the `checkcast` bytecode instruction. The JVM verifies the concrete class of the retrieved object, not the existence of the original parameterized type.

### 2.2.4 Bridge Methods

Erasure can break overriding relationships at the JVM level. Given `Node<T>.get():T` overridden by `StringNode.get():String`, erasure reduces the parent signature to `get():Object`; the compiler therefore generates a synthetic bridge method, distinguishable from the covariant override only at the bytecode level by its descriptor `()Ljava/lang/Object;`, which invokes the specific `String get()` implementation and returns its result as `Object`, in order to preserve polymorphic dispatch through the erased parent signature.

### 2.2.5 Primitive Types

Because erased parameters are represented as reference types, primitive instantiation (`List<int>`) is disallowed; wrapper types (`List<Integer>`) are required instead, so that $int \rightarrow Integer \rightarrow Object$, with boxing and unboxing performed on insertion and retrieval.

### 2.2.6 Comparison with C++ Templates

Java preserves abstraction through erasure, sharing one runtime representation ($List<T> \rightarrow List<Object>$) across all instantiations; C++ preserves abstraction through specialization, generating independent representations ($Vector<int>$, $Vector<double>$, ...) for each argument. Java avoids multiple generated implementations at the cost of runtime visibility of generic parameters, requiring casts, boxing, and bridge methods; C++ retains concrete type information at the cost of increased code size.

## 2.3 Performance Characteristics

The performance profile of Java generics follows directly from type erasure: a single shared implementation executes for all parameterizations, so the JVM operates on `Object` references rather than specialized, type-tailored code.

### 2.3.1 Memory Representation and Pointer Indirection

Generic structures operate primarily on references rather than direct values. `ArrayList<Integer>.get(0)` does not store the integer directly; it dereferences an `Integer` object on the heap. C++ templates, by contrast, allow direct contiguous storage (`std::vector<int>`), avoiding this indirection and improving cache locality.

### 2.3.2 Boxing and Unboxing Overhead

Since generic parameters must be reference types, primitive values require wrapper objects: `values.add(5)` requires boxing ($int \rightarrow Integer$), and retrieval requires unboxing. This introduces allocation, additional memory accesses, and conversion costs absent from C++, which manipulates primitives natively within instantiations.

### 2.3.3 Casts and Bridge Methods at Runtime

Type erasure requires inserted `checkcast` verification on retrieval, and synthetic bridge methods to preserve overriding under erasure; both introduce a small runtime cost, though modern JIT compilers frequently eliminate redundant checks and inline bridge calls.

### 2.3.4 Cache Locality

Reference-based storage scatters logical elements across the heap, increasing cache misses during traversal relative to the contiguous value layout achievable through C++ template instantiation.

### 2.3.5 Adaptive JIT Optimization

The HotSpot JVM mitigates some of these costs through runtime profiling: devirtualization and inline caching reduce dispatch overhead, while escape analysis enables scalar replacement of non-escaping boxed values, eliminating their allocation in specific, provably local cases. These optimizations reduce boxing overhead in favorable cases but do not restore the contiguous, specialized storage of generic containers, which retain their reference-based internal representation regardless

of JIT optimization, unlike the ahead-of-time specialization guaranteed by C++ monomorphization.

### 2.3.6 Comparative Summary

| Characteristic | Java Generics | C++ Templates |
|---|---|---|
| Representation | Type erasure, shared runtime class | Compile-time monomorphization |
| Primitives | Boxing/unboxing required | Direct native representation |
| Memory access | Reference indirection | Direct contiguous storage |
| Optimization | Adaptive JIT | Ahead-of-time compiler |
| Cache locality | Reduced by object layout | Improved via value layout |

Table 2.1: Runtime performance comparison between Java Generics and C++ templates.

## 2.4 Type Safety

Java generics extend the static type system with parameterized types, guaranteeing that substitutions of a type parameter cannot introduce operations violating the constraints established at declaration.

### 2.4.1 Static Type Checking

Given `class Box<T> { T value; void set(T v); T get(); }`, the compiler treats `Box<String>` as a type in which every use of `T` must be compatible with `String`; given `Box<String> box`, the call `box.set(10)` is therefore rejected statically, since `Integer` does not satisfy the parameterization.

### 2.4.2 Bounded Type Parameters

Bounds such as `<T extends Comparable<T>` restrict admissible substitutions to types providing the required operations (here, `compareTo`), and the compiler rejects any instantiation that fails to satisfy the bound.

### 2.4.3 Erasure and Preservation of Safety

Erasure does not weaken these guarantees: correctness is established before erasure occurs. For `List<String> names; names.add("Alice"); String s = names.get(0);`, the compiler verifies both insertion and retrieval against `String` before the generic parameter is discarded from the representation. Java therefore separates *static type correctness*, guaranteed by generic checking, from *type representation*, determined only afterward by erasure.

### 2.4.4 Comparison with C++ Reification

C++ templates preserve type information through compile-time specialization: each instantiation is checked against its concrete type directly, as in `Box<int> box; box.set("text");`, rejected because `int` does not accept a string. Java instead verifies every substitution before erasure removes the parameter. The distinction is therefore not whether type safety is provided, but at which stage type information is consumed: Java achieves safety through static verification prior to erasure, while C++ achieves safety through compile-time specialization of concrete types, checked independently for each instantiation, with no corresponding generic entity surviving into the running program.

## 2.5 Reflection and Runtime Type Information

Java's type-erasure model determines the boundaries of its runtime reflection capabilities: since all parameterizations of a generic declaration share one erased runtime class, reflection can only observe the erased representation, never the original concrete type argument.

### 2.5.1 The `Class` Object and Erasure

`List<String>` and `List<Integer>` both correspond to the same runtime class, `java.util.List`; the JVM retains generic declaration metadata in the class file's `Signature` attribute, but individual object instances carry only the erased type.

### 2.5.2 RTTI Limitations

Because parameterized arguments are erased, expressions such as `obj instanceof List<String>` are disallowed by the Java Language Specification, and `getClass()` returns identical class objects for `ArrayList<String>` and `ArrayList<Integer>`: the JVM has no runtime information capable of distinguishing them.

### 2.5.3 Type Tokens

To recover runtime type information despite erasure, Java libraries pass an explicit `Class<T>` object alongside the generic value (a *type token*), as in `Box(Class<T> type, T value)`, allowing frameworks such as serializers and dependency injectors to query the concrete type independently of the erased generic parameter.

### 2.5.4 Comparison with C++ Template RTTI

C++ monomorphization generates an independent concrete type and independent RTTI metadata– for every specialization, so `typeid` and `dynamic_cast` can distinguish `Vector<int>` from `Vector<double>` directly. Java's erasure model instead collapses all parameterizations into a single `Class` object, so reflection exposes only the identity of the generic declaration after erasure, never the parameterized instantiation present in the source. This contrast illustrates the underlying trade-off: C++ monomorphization preserves runtime type identity per specialization as a by-product of compile-time code generation, while Java erasure minimizes runtime type diversity at the cost of that same visibility.

# Chapter 3

# C# Generics

C# implements parametric polymorphism through *generics*, adopting a runtime-oriented model in which the Common Language Runtime (CLR) preserves generic type parameters as metadata and instantiates concrete specializations only when required during execution, rather than during source compilation. This chapter examines when this runtime instantiation occurs, how the CLR combines specialization for value types with code sharing for reference types, and how this reified, JIT-driven model influences performance, static type safety, and the extensive runtime reflection capabilities it enables.

## 3.1 Instantiation Time

C# generics adopt a runtime-oriented instantiation model, distinct from both the compile-time monomorphization of C++ templates and the compile-time erasure of Java generics. Generic definitions are compiled ahead of execution, but their concrete executable representation is generated only at runtime by the Common Language Runtime (CLR) and its Just-In-Time (JIT) compiler.

### 3.1.1 Generic Definition

The C# compiler translates a generic declaration into Common Intermediate Language (CIL), preserving type parameters as metadata rather than expanding them:

```
class Container<T>
{
    public T value;
}
```

No specialized implementations such as `Container<int>` or `Container<double>` are generated at this stage; the assembly contains a single open generic definition.

### 3.1.2 Instantiation Trigger

Instantiation occurs when the CLR resolves a concrete constructed type during execution:

**Example.**

```
Container<int> c;
```

The CLR resolves `Container<T>` together with the argument `int`, producing the constructed type `Container<int>`. Only when native code is required does the JIT compiler translate the corresponding CIL into machine instructions.

The instantiation timeline is: compiler emits generic CIL → assembly loaded by the CLR → constructed type requested at execution → CLR materializes it → JIT generates native code when required.

### 3.1.3 Hybrid Specialization Strategy

The JIT does not always generate an independent implementation per instantiation. For reference types, since all references share a compatible representation, a single implementation can be reused across `List<string>`, `List<object>`, `List<MyClass>`. For value types, differing memory layouts require the JIT to generate separate specializations: `List<int>` and `List<double>` trigger distinct JIT specialization events. The model is therefore hybrid: runtime instantiation always occurs, but reference types favor code sharing while value types trigger specialization.

### 3.1.4 Comparison with C++ and Java

C++ performs specialization ahead of execution (Template Definition → Compile-Time Instantiation → Native Code → Execution); Java erases parameters at compile time with no runtime instantiation event (Generic Source → Erasure → Bytecode → Execution); C# instead delays specialization until execution (Generic Definition → CIL → Runtime Instantiation → JIT → Execution).

| Language | Instantiation Time | Specialization Phase |
|---|---|---|
| C++ | Compile time | Compiler monomorphization |
| C# | Runtime | CLR/JIT instantiation |
| Java | Compile time | Type erasure, no specialization |

The defining property of C# generics is therefore the placement of the instantiation event at runtime, with the JIT compiler acting as the specialization mechanism.

## 3.2 Runtime Representation

C# generics implement parametric polymorphism through *reification*: generic type information is preserved after compilation and remains available to the execution environment, combined with selective JIT specialization. The pipeline is: C# Source → CIL + Metadata → CLR Generic Instantiation → JIT Compilation → Native Code.

### 3.2.1 CIL Metadata Representation

The compiler generates CIL and CLI metadata (ECMA-335) preserving explicit generic declarations: `List<T>` is stored as an open generic type definition, not a concrete implementation. Dedicated metadata tables – `GenericParam`, `GenericParamConstraint`, `TypeSpec`, `MethodSpec` – let the CLR reconstruct the relationship between a generic definition and its concrete arguments, e.g. `List<T>` → `List<int>`.

### 3.2.2 Runtime Instantiation and JIT Compilation

A generic definition $G\langle T\rangle$ becomes a runtime constructed type $G\langle U\rangle$ only when required; `Dictionary<TKey,TValu` may generate instantiations such as `Dictionary<int,string>` or `Dictionary<Guid,Object>` at runtime, with the JIT generating native code according to the supplied arguments.

### 3.2.3 Value-Type Specialization

Because each value type has its own size and memory layout, the JIT generates separately specialized native code for each such instantiation: $JIT(\texttt{List<int>}) = NativeCode_{int}$, $JIT(\texttt{List<double>}) = NativeCode_{double}$, each operating directly on the corresponding representation.

### 3.2.4 Reference-Type Code Sharing

Because reference types share a uniform object-reference representation, CLR implementations such as the Microsoft CLR can reuse a single compiled implementation, $NativeCode_{reference}$, across instantiations such as `List<string>` and `List<object>`, supplying type-specific information through runtime execution-engine structures rather than duplicating code.

### 3.2.5 Runtime Generic Dictionaries

Shared implementations rely on runtime generic dictionaries/contexts to resolve type descriptors, method addresses, field layouts, and interface implementations at execution time – a dictionary-passing-like mechanism with no equivalent in C++ (resolved before execution) or Java (erased before execution).

### 3.2.6 Runtime Type Identity

Constructed generic types retain distinct runtime identity: $RuntimeType(\texttt{List<int>}) \neq RuntimeType(\texttt{List<st}$ unlike Java where both erase to the same `List`. C++ specializations can participate in RTTI as concrete types only within polymorphic class hierarchies (i.e., containing at least one virtual function); the original template definition itself is never preserved as a runtime entity, regardless of polymorphism.

### 3.2.7 Architectural Comparison

| Feature | C# Generics | Java Generics | C++ Templates |
|---|---|---|---|
| Generic model | Reification | Type Erasure | Monomorphization |
| Metadata | Preserved in CLR | Removed at execution | Not available |
| Instantiation | Runtime/JIT | Compile-time only | Compile-time |
| Code generation | Specialize + share | Single erased code | Native specialization |
| Runtime identity | Arguments preserved | Arguments removed | Concrete types only |

Table 3.1: Comparison of generic runtime representation models.

## 3.3 Performance Characteristics

C# generics combine runtime reification with JIT compilation: type parameters survive into CIL metadata, and the execution strategy depends on whether the argument is a value type (specialized native code) or a reference type (shared native code):

$$\text{Generic<T> execution} = \begin{cases} \text{specialized code} & T \text{ value type} \\ \text{shared code} & T \text{ reference type} \end{cases}$$

This hybrid model, as implemented by common CLR implementations such as the Microsoft CLR, sits between C++'s full specialization and Java's fully erased execution.

### 3.3.1 Value-Type Instantiations

For value-type instantiations, elements are stored directly inside the containing structure. A `Buffer<Vector3>` array stores contiguous structs rather than references, enabling sequential cache-friendly access and avoiding boxing entirely, execution cost across the whole collection approximates $T_{value} = N(C_{load} + C_{operation})$, with per-element cost closer to specialized C++ code than to Java's boxed collections.

### 3.3.2 Reference-Type Instantiations

For reference-type instantiations, e.g. `Buffer<VectorClass>`, the array stores pointers rather than objects, requiring an additional dereference per access: $T_{reference} = N(C_{load\ ref} + C_{pointer\ chase} + C_{operation})$, reducing spatial locality relative to value-type storage, though C#'s avoidance of boxing, unlike Java, applies specifically to value-type instantiations such as those discussed above, not to reference-type instantiations, where none of the three languages requires boxing.

### 3.3.3 Comparison with C++ Monomorphization

C++ generates a fully independent specialization per instantiation ahead of execution (`Buffer<float>`, `Buffer<double>`), maximizing optimization opportunities (static dispatch, inlining) at the cost of increased code size. C# avoids this duplication for reference types while still specializing value types via the JIT, trading some ahead-of-time optimization for reduced code bloat.

### 3.3.4 Comparison with Java Type Erasure

Java's single erased implementation forces primitive values through boxing/unboxing ($int \rightarrow Integer \rightarrow reference$), adding $T_{Java} = T_{operation} + C_{boxing} + C_{indirection}$ relative to C#, which avoids boxing for value-type generics entirely.

### 3.3.5 Comparative Summary

| | C++ Templates | C# Generics | Java Generics |
|---|---|---|---|
| Representation | Compile-time specialization | Runtime reification | Type erasure |
| Value storage | Direct | Direct | Usually boxed |
| Code generation | AOT monomorphization | JIT specialization | Shared erased code |
| Cache locality | Very high | High (value types) | Lower (indirection) |
| Runtime overhead | Minimal | Low | Higher for primitives |

C# therefore occupies an intermediate position: near-C++ efficiency for value-oriented workloads, with a more flexible runtime model than Java's erased generics.

## 3.4 Type Safety

C# generics preserve type safety through static verification of generic usage, ensuring every instantiation respects the constraints declared by the generic abstraction.

### 3.4.1 Static Checking of Generic Parameters

Given `class Box<T> { private T value; public void Set(T v) { value = v; } public T Get() { return value; } }`, the compiler verifies that all values passed to `Set` match the chosen instantiation: `Box<int> numbers = new Box<int>(); numbers.Set("text");` is rejected because `Box<int>` requires every stored value to be an integer.

### 3.4.2 Generic Constraints

A parameter without constraints cannot assume arbitrary members are available: `value.SpecialOperation()` inside `Processor<T>` is rejected unless a constraint such as `where T : IDisposable` establishes that every valid substitution provides `Dispose()`. Constraints act as additional typing rules restricting admissible substitutions, verified statically rather than at runtime.

### 3.4.3 Variance and Safe Subtyping

Mutable generic containers are treated as invariant to prevent unsafe insertion: `List<object> objects = strings;` (from `List<string>`) is rejected. Read-only interfaces permit covariance instead: `IEnumerable<object> objects = strings;` is valid because values are only produced, never consumed, through the interface.

### 3.4.4 Reification and Preserved Type Information

`Box<int>` and `Box<string>` are checked as distinct constructed types already at compile time, independently of reification; because C# is reified, this distinction also persists at runtime, so their invariants cannot cross-contaminate at either stage. This contrasts with Java, where generic correctness is verified before erasure removes the parameter from the runtime representation, and with C++, where correctness is verified per template substitution during instantiation.

### 3.4.5 Type Safety Guarantees

C# generics guarantee that: every instantiation satisfies its declared constraints; operations on parameters are valid for every permitted substitution; variance rules restrict subtyping to safe cases; the association between generic definitions and concrete arguments is preserved; and invalid conversions between incompatible constructed types are rejected statically.

## 3.5 Reflection and Runtime Type Information

The CLR implements a reified generic model: generic declarations, constraints, and instantiation relationships are encoded in assembly metadata and remain available during execution, exposed through `System.Type`.

### 3.5.1 Open and Closed Generic Types

`typeof(List<>)` represents an open generic definition; `typeof(List<int>)` represents a closed constructed type; `typeof(List<int>).GetGenericArguments()` retrieves the concrete runtime argument. Reflection therefore operates on actual runtime type entities rather than erased or compile-time-only abstractions.

### 3.5.2 Runtime Inspection Example

```
Type t = instance.GetType();
if (t.IsGenericType)
{
    Type def = t.GetGenericTypeDefinition();
    Type[] args = t.GetGenericArguments();
}
```

For a `List<int>` instance, this recovers both the generic definition `List<>` and the concrete argument `int`: `List<int>` $\rightarrow$ `List<>` + `int`.

### 3.5.3 Comparison with C++ RTTI

C++ monomorphization resolves template parameters at compile time; standard RTTI (`typeid`, `std::type_info`) can identify the resulting concrete type only when the class hierarchy is polymorphic (i.e., contains at least one virtual function), and even then cannot query the original template argument as runtime metadata, the template/parameter relationship exists only at compile time.

### 3.5.4 Comparison with Java Reflection

Java erasure removes generic parameters from runtime object identity: two `ArrayList` instances built from different type arguments share the same runtime class. Reflection can recover declaration-level generic signatures (via the `Signature` attribute) but cannot recover the concrete argument used by an arbitrary instance.

### 3.5.5 Comparative Summary

C# reifies generic parameters, exposing both open definitions and closed constructed types through `System.Reflection`; C++ resolves parameters at compile time, leaving RTTI able to identify only concrete specialized types, and only within polymorphic class hierarchies; Java erases parameters from runtime identity, leaving reflection able to inspect only declaration-level metadata. The CLR model therefore keeps generic structure part of the runtime type universe, while C++ and Java represent the two opposite design points, compile-time expansion without runtime metadata, and runtime erasure of generic identity.

# Chapter 4

# Metaprogramming

Metaprogramming is a programming technique in which a program can generate, analyze, or transform program entities rather than manipulating only runtime data. Its main objective is to move part of the computation to an earlier stage of the software lifecycle, such as compilation, or to enable dynamic modification and inspection of program structures during execution. The specific implementation of metaprogramming depends on the language compilation model and runtime architecture.

In **C++**, metaprogramming is primarily performed at compile time through template metaprogramming and constant expression evaluation mechanisms such as `constexpr`. Template instantiation allows the compiler to generate specialized program entities during translation, while `constexpr` enables the evaluation of deterministic expressions before runtime. C++ metaprogramming is therefore tightly integrated with the compiler and the static type system.

In **Java**, metaprogramming is mainly based on runtime mechanisms provided by the Java Virtual Machine (JVM), particularly reflection, annotations, dynamic proxies, and bytecode generation. Compile-time metaprogramming is more limited and is mainly supported through annotation processors. Because Java generics are implemented through type erasure rather than instantiation-time substitution, no specialized version of generic code is generated per concrete type, which prevents type-based compile-time transformations; as a further consequence of the same erasure process, generic type parameters are also generally unavailable in ordinary runtime objects.

In **C#**, metaprogramming combines compile-time and runtime approaches. Compile-time mechanisms include source generators, which produce additional code during compilation, while runtime mechanisms are provided by reflection, attributes, expression trees, and dynamic code generation facilities of the Common Language Runtime (CLR). Unlike Java, C# preserves generic type information at runtime through reified generics, enabling more extensive runtime type inspection.

The main difference between these languages therefore lies in *when* metaprogramming is performed: C++ emphasizes compile-time computation, Java primarily relies on runtime introspection, and C# provides a hybrid model combining compile-time generation with runtime analysis.

## 4.1 Compile-Time Computation in C++

C++ supports compile-time computation through two mechanisms relying on different compiler rules: recursive template instantiation, which performs computation through generation and substitution of template entities, and `constexpr` evaluation, which relies on direct interpretation of valid constant expressions. The following examples assume C++17 or later.

### 4.1.1 Recursive Template Metaprogramming

Traditional template metaprogramming performs computation by recursively instantiating templates, each specialization representing a distinct compile-time entity:

```
template<unsigned N>
struct Factorial
{
    static constexpr unsigned value =
        N * Factorial<N - 1>::value;
};

template<>
struct Factorial<0>
{
    static constexpr unsigned value = 1;
};

constexpr unsigned result = Factorial<6>::value;
```

Resolving `Factorial<6>::value` recursively instantiates `Factorial<6>` $\rightarrow$ `Factorial<5>` $\rightarrow \cdots \rightarrow$ `Factorial<0>`, the last specialization terminating the recursion. The compiler reduces the instantiated chain to the constant 720 during semantic analysis; no runtime recursive execution is generated, since template arguments are fully known during translation.

The same mechanism applies to computations with multiple recursive dependencies. A related but structurally different case is the Fibonacci sequence, whose naive recursive definition requires each intermediate value to be reached through multiple paths in the dependency tree, unlike the single linear chain produced by `Factorial`:

```
template<unsigned N>
struct Fibonacci
{
    static constexpr unsigned value =
        Fibonacci<N - 1>::value + Fibonacci<N - 2>::value;
};

template<> struct Fibonacci<0> { static constexpr unsigned value = 0; };
template<> struct Fibonacci<1> { static constexpr unsigned value = 1; };
```

Instantiating `Fibonacci<10>::value` generates this dependency tree implementing $F(n) = F(n-1) + F(n-2)$ until reaching the terminating specializations, yielding the constant 55 before code generation; redundant re-instantiation of shared intermediate values such as `Fibonacci<8>` is avoided only because the compiler caches each distinct specialization once it has been instantiated; the recursive structure exists only as a compile-time artifact.

### 4.1.2 `constexpr` Function Evaluation

Since C++11, the `constexpr` specifier allows ordinary functions to participate in constant expression evaluation without generating specialized template entities:

```
constexpr unsigned factorial(unsigned n)
{
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

constexpr unsigned result = factorial(6);
```

Because `result` is declared `constexpr`, the initializer must be evaluated during translation: the compiler's constant expression evaluator interprets the function body directly, producing 720, which is substituted into the program representation. The final executable contains no recursive runtime call.

Both mechanisms complete the transformation *compile-time expression* $\rightarrow$ *constant evaluation* $\rightarrow$ *generated representation* before runtime execution begins; template metaprogramming performs this through instantiation and substitution, while `constexpr` performs it through direct evaluation

of the function body against the constant-expression rules of the standard.

## 4.2 Compile-Time Overload Selection: SFINAE and Concepts

SFINAE (*Substitution Failure Is Not An Error*) enables compile-time selection of function overloads by controlling whether a template declaration participates in overload resolution. The following example uses `std::enable_if` to select an implementation according to a compile-time property of the argument type:

```
template <typename T>
typename std::enable_if<std::is_integral<T>::value>::type
process(T value) { std::cout << "Integral: " << value; }

template <typename T>
typename std::enable_if<std::is_floating_point<T>::value>::type
process(T value) { std::cout << "Floating-point: " << value; }
```

During argument substitution, the compiler attempts to instantiate each candidate. For `process(42)`, $T = int$ makes `std::enable_if<true>::type` well-formed, so the integral overload remains viable; the floating-point overload instead produces `std::enable_if<false>`, which lacks a nested `type` member, so substitution fails and the overload is silently discarded rather than triggering a compilation error. This behavior applies only within the immediate context of substitution during template argument deduction.

C++20 Concepts express the same constraints explicitly rather than through substitution failure:

```
#include <concepts>
template <typename T> requires std::integral<T>
void process(T value) { std::cout << "Integral: " << value; }

template <typename T> requires std::floating_point<T>
void process(T value) { std::cout << "Floating-point: " << value; }
```

The compiler evaluates each `requires` clause before overload resolution, discarding candidates whose constraints are not satisfied – providing the same compile-time specialization behavior as SFINAE while stating the type requirements directly rather than relying on incidental substitution failure.

## 4.3 Variadic Templates and Related C++ Techniques

Variadic templates, introduced in C++11, allow a template to accept an arbitrary number of type or non-type arguments through a parameter pack, enabling generic components whose arity is not fixed in advance:

```
template <typename T>
void print(T value) { std::cout << value << ' '; }

template <typename T, typename... Ts>
void print(T first, Ts... rest) {
    std::cout << first << ' ';
    print(rest...);
}
```

The parameter pack `Ts...` represents an arbitrary type sequence; the compiler recursively expands it during instantiation until the single-argument base case is reached, with the entire expansion resolved at compile time through overload resolution.

### 4.3.1 Template Parameters

Template template parameters allow a template to receive another template as an argument, enabling higher-order generic programming over families of compatible templates rather than over individual types:

```cpp
template <typename T, template <typename, typename...> class Container>
class Stack {
    Container<T> data;
public:
    void push(const T& value) { data.push_back(value); }
};

Stack<int, std::vector> a;
Stack<int, std::list> b;
```

The variadic `typename...` accounts for the additional defaulted template parameters of standard containers such as `std::vector` and `std::list` (e.g. the allocator type), which a template template parameter declared with a single `typename` would not match. The compiler substitutes the supplied template argument for `Container` and instantiates the resulting specialization, preserving compile-time type checking across the abstraction.

### 4.3.2 Curiously Recurring Template Pattern (CRTP)

CRTP is a template-based inheritance technique in which a derived class passes itself as the template argument to its base class, enabling static polymorphism through compile-time binding rather than runtime dynamic dispatch:

```cpp
template <typename Derived>
class Interface {
public:
    void execute() { static_cast<Derived*>(this)->implementation(); }
};

class Concrete : public Interface<Concrete> {
public:
    void implementation() { std::cout << "Concrete implementation"; }
};
```

The base template is instantiated with the derived type, so calls to derived operations are resolved during compilation; the compiler generates a concrete implementation for each participating derived type, without requiring a runtime virtual interface.

## 4.4 Bounded Polymorphism in Java: Constraints Without Substitution

Java generics express bounded polymorphism through explicit type constraints restricting admissible type arguments:

```java
public class Calculator<T extends Number> {
    public double square(T value) {
        return value.doubleValue() * value.doubleValue();
    }
}
```

The bound `T extends Number` performs compile-time checking that only types satisfying the declared relationship may instantiate the class. However, this expresses only the operations guaranteed by the bound; it cannot introduce conditional compile-time branching based on the concrete identity of `T`, because Java generics are implemented through type erasure rather than instantiation-time substitution. Since no specialized version of generic code is generated per concrete type, a method such as

```java
public static <T> void process(T value) {
    if (value instanceof Integer) {
        // still cannot call Integer-specific methods on 'value'
        // without an explicit cast, since the declared type of
        // value remains T, not Integer
```

```
    }
}
```

cannot be compiled into distinct versions depending on `T`: even after a runtime `instanceof` check, the static type of `value` remains `T`, so any type-specific member access still requires an explicit cast performed by the programmer, not a compiler-generated specialization. Any type-dependent behavior must therefore rely on mechanisms available only after erasure, such as runtime type information, rather than on compile-time substitution analysis as in SFINAE (Section 4.2).

## 4.5 Constrained Generics in C#: Approximating Type Traits

C# does not provide a direct equivalent of SFINAE or compile-time type traits, since its generic constraints express bounded polymorphism rather than substitution-based overload resolution. However, `where` clauses combined with static abstract interface members (introduced in .NET 7) allow a restricted form of trait-oriented generic programming.

Where a C++ trait pattern conditionally enables an overload via SFINAE:

```
template<typename T>
auto add(T a, T b) -> std::enable_if_t<std::is_arithmetic_v<T>, T>
{ return a + b; }
```

the closest C# approximation constrains the parameter directly, using the standard-library `INumber<T>` interface introduced together with static abstract interface members in .NET 7:

```
static T Add<T>(T a, T b) where T : INumber<T>
{
    return a + b;
}
```

This works because `INumber<T>` is itself defined in terms of static abstract interface members, which allow a trait-like contract to be expressed directly:

```
public interface IAddable<TSelf> where TSelf : IAddable<TSelf>
{
    static abstract TSelf operator +(TSelf x, TSelf y);
}
```

The compiler validates the presence of required static members during generic instantiation, conceptually resembling C++ concepts, though more restricted. This validation itself occurs at compile time, independently of runtime representation; separately, .NET generics also use runtime reification, preserving type arguments and constraints in CLR metadata for runtime inspection, unlike Java's erased generics. Nevertheless, C# constraint violations are ordinary compilation errors, not substitution failures that selectively remove candidates during overload resolution – the language provides no general compile-time reflection over type structure or unrestricted compile-time computation.

## 4.6 Instantiation-Time Checking and Structural Capability Detection

The distinction explored in the previous two sections follows from a more general property: C++ templates perform semantic checking *after* argument substitution, during instantiation, so a dependent expression's validity need not be known until a concrete type is supplied. Java and C# generics instead require declarations to be type-correct independently of any future instantiation, permitting only operations guaranteed by the declared bounds.

This lets C++ detect structural capabilities – whether a type happens to provide a given member – without an explicit interface:

```
template <typename T>
auto process(T value) -> decltype(value.serialize(), void())
```

```cpp
{ value.serialize(); }

template <typename T>
void process(T) { /* fallback */ }
```

If `T` provides `serialize()`, the first overload is selected; otherwise substitution of `value.serialize()` fails and SFINAE falls back to the second overload – capability detection with no inheritance relationship involved.

Java and C# cannot express this directly, since generic bodies must already be valid for every admissible argument:

```java
static <T> void process(T value) {
    value.serialize(); // compilation error: no guarantee T has serialize()
}
```

The only available option is to encode the requirement explicitly through bounded polymorphism:

```java
interface IManualSerializable { void Serialize(); }

static void Process<T>(T value) where T : IManualSerializable
    => value.Serialize();
```

but such a constraint must be declared ahead of time and cannot be inferred from the structural properties of an arbitrary type. The limitation follows directly from declaration-time checking: generic bodies are validated once, for all admissible arguments, whereas C++ defers validity to each individual instantiation.

## 4.7 External Metaprogramming Mechanisms in Java and C#

Because Java and C# generics do not integrate compile-time program generation into their type systems, both languages rely on auxiliary mechanisms, external to the generic abstraction itself, to approximate capabilities native to C++ templates.

### 4.7.1 Java: Reflection, Annotation Processing, and Code Generation

Java relies on runtime *reflection* to recover structural information unavailable through generic operations, inspecting classes, methods, fields, and annotations from metadata stored in class files after compilation. Reflection is runtime introspection: it does not participate in generic instantiation or compiler-driven specialization.

Compile-time generation is instead handled by *annotation processors* (Java Annotation Processing API), which run during compilation, analyze annotated source elements through the compiler's language model API, and generate auxiliary source files. Libraries such as Lombok achieve a similar effect through direct AST manipulation, injecting methods or constructors that developers would otherwise write by hand; unlike standard annotation processors, Lombok relies on internal, unsupported compiler APIs to modify existing source rather than only generating new files, which is why modern JDK versions require explicit module-access flags to permit it. In both cases, these are external compiler extensions: the generic type system remains responsible only for type abstraction and checking, while generation is performed by a separate framework rather than by generic substitution itself.

### 4.7.2 C#: Source Generators and T4 Templates

C# offers more integrated compile-time generation through Roslyn *source generators*, which run during compilation, analyze syntax trees and semantic models, and emit additional C# source that becomes part of the compilation unit – e.g. serialization code or boilerplate implementations. Despite operating inside the compiler pipeline, source generators do not participate in generic instantiation: generic declarations are still processed by the ordinary CLR generic model, while generated code is simply additional input produced by a separate transformation phase.

*T4 (Text Template Transformation Toolkit)* templates operate earlier still, as an independent preprocessing stage that emits source files from explicit transformation rules before ordinary compilation begins.

In both Java and C#, generics describe reusable abstractions within the language type system, while reflection, annotation processors, source generators, and T4 templates remain auxiliary transformation layers around the compiler or runtime. C++ templates, by contrast, unify parametric polymorphism and compile-time generation in a single construct, where template parameters directly drive compiler substitution and specialization.

## 4.8 Summary: Metaprogramming Capabilities Across the Three Models

| **Capability** | **C++ Templates** | **Java Generics** | **C# Generics** |
|---|---|---|---|
| Compile-time computation | **Supported** – template metaprogramming and `constexpr` evaluate computations during compilation. | **Not supported** – type erasure removes generic specialization entirely. | **Partially supported** – constraints allow static reasoning but not general compile-time computation. |
| Conditional selection by type properties | **Supported** – SFINAE, type traits, and Concepts select implementations by compile-time properties. | **Not supported** – erasure removes concrete type properties before runtime. | **Partially supported** – constraints select by declared relationships, not arbitrary predicates. |
| Code generation per type | **Supported** – monomorphization generates independent code per instantiation. | **Not supported** – erasure yields one shared implementation for all instantiations. | **Supported for value types, shared for reference types** – CLR reification specializes value-type instantiations via JIT. |
| Runtime type introspection | **Partially supported** – RTTI exposes concrete object types only within polymorphic class hierarchies (i.e., at least one virtual function), not template parameters. | **Partially supported** – reflection sees declaration-level generic metadata, not runtime arguments. | **Supported** – CLR metadata preserves generic arguments for runtime inspection. |

Table 4.1: Practical limits of metaprogramming capabilities in C++, Java, and C# generics.